Self Consistency

Agentic AI

- Self-consistency is a decoding/reasoning strategy where, instead of generating a single reasoning path to an answer, the model generates **multiple independent reasoning paths** for the same problem and then selects the final answer via **majority vote (or aggregation)** across those paths
- It was introduced as an improvement over standard Chain-of-Thought (CoT) prompting
- The core insight: complex reasoning problems often admit multiple valid lines of reasoning that converge on the same correct answer, while incorrect reasoning tends to scatter across different wrong answers
- **Core Idea (Origin)**
  - Comes from the paper **"Self-Consistency Improves Chain of Thought Reasoning in Language Models"** (Wang et al., Google Research, 2022)
  - Standard CoT: greedy decoding - one reasoning chain - one answer
  - If that one chain has an error, the final answer is wrong
  - Self-consistency: sample **k** diverse reasoning chains (via temperature/top-p sampling, not greedy decoding) - extract the final answer from each - take the answer that appears most frequently

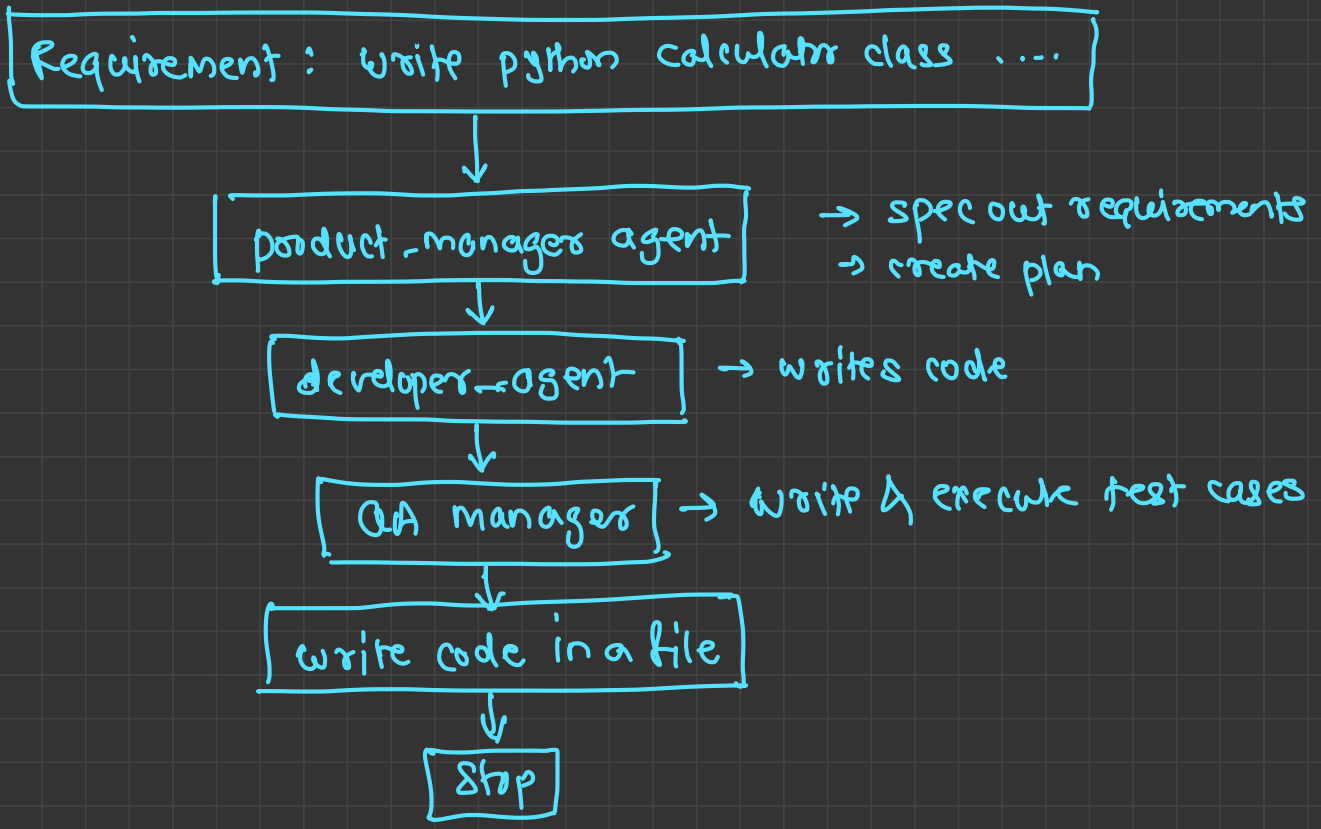
How it works?

- **Prompt construction**
  - Use a CoT-style prompt (few-shot exemplars showing step-by-step reasoning, or a zero-shot "think step by step" instruction)
- **Diverse sampling**
  - Instead of decoding once with temperature=0, sample the model N times (e.g., N = 5, 10, 20, 40) with temperature > 0 to get varied reasoning paths
- **Independent reasoning**
  - Each sample independently reasons through the problem in its own way — different decomposition, different intermediate steps, different order of operations
- **Answer extraction**
  - Parse out the final answer from each of the N generated chains
- **Aggregation / voting**
  - Take the majority answer (mode) across the N samples as the final output
  - Ties can be broken by confidence scores, weighted voting, or re-prompting.

- Question ⟶
  - [Sample 1: reasoning path A ⟶ answer Y]
  - [Sample 2: reasoning path B ⟶ answer X]
  - [Sample 3: reasoning path C ⟶ answer Y]
  - [Sample 4: reasoning path D ⟶ answer X]
  - [Sample 5: reasoning path E ⟶ answer X]
- Majority vote ⟶ Answer X

Why It Works

- **Error decorrelation**
  - Wrong reasoning tends to fail in different ways each time (different mistaken assumptions, arithmetic slips, missed steps), so wrong answers rarely cluster
- **Correct reasoning converges**
  - Multiple valid reasoning strategies for the same problem tend to arrive at the same correct answer, so correct answers cluster
- It's essentially an ensembling technique at the reasoning-path level, analogous to bagging/ensemble methods in classical ML, but applied to generated reasoning traces instead of trained models
- It doesn't require any additional training, fine-tuning, or auxiliary verifier model — it's purely an inference-time strategy


Requirement : write python calculator class ....
product_manager agent
→ spec out requirements
→ create plan
developer_agent
→ writes code
QA manager
→ write & execute test cases
write code in a file
Stop

Where It Fits in Agentic AI Design Patterns



- In the broader landscape of agentic patterns, self-consistency sits in the **reasoning/reflection family** alongside:
  - Chain-of-Thought: Single linear reasoning trace
  - Self-Consistency: Multiple sampled reasoning traces + majority vote
  - Tree-of-Thought: Explicit branching + search/backtracking over reasoning states
  - ReAct: Interleaved reasoning + tool-use actions
  - Reflection / Self-Refine: Iterative self-critique and revision of one trace
  - Debate: Multiple agent personas argue and cross-examine
- Self-consistency is distinct from Tree-of-Thought in that it does **not** require explicit search, backtracking, or evaluation of intermediate states
- The paths run to completion independently and are only compared at the final-answer stage
- It's cheaper and simpler to implement than ToT, but less powerful for problems needing mid-reasoning correction

Where It's Most Effective



- Arithmetic reasoning (word problems)
- Commonsense reasoning
- Symbolic reasoning tasks with a discrete, verifiable answer space
- Tasks where the answer space is small and discrete (multiple choice, numeric, short factual) — voting is straightforward
- Less effective / harder to apply for open-ended generation (long-form writing, code generation with many valid solutions) because "majority vote" over free text is ill-defined — though variants exist (e.g., clustering similar outputs, or using execution-based voting for code, like running test cases)

Cost / Latency Trade-offs



- N× the inference cost and latency of a single CoT call (you're running N samples instead of 1)
- Diminishing returns: accuracy gains typically saturate around N = 10–40 samples depending on task difficulty
- Common mitigation in agentic systems
  - Run samples in parallel (not sequential) to bound latency
  - Use a cheaper/smaller model for the N sampled paths and a larger model only for final verification/tie-breaking
  - Adaptive sampling: start with a small N, only sample more if there's no clear majority (early-stopping / dynamic self-consistency)

Variants and Extensions

- **Universal Self-Consistency**
  - uses the LLM itself (rather than exact-match voting) to select the best answer among the N generations, useful when answers are free-form text rather than short discrete outputs
- **Self-Consistency + ToT**
  - sample multiple full trees / search rollouts, then vote
- **Self-Consistency with tools**
  - each sampled path can include different tool-call sequences; vote over final grounded answers rather than raw reasoning
- **Confidence-weighted self-consistency**
  - incorporate the model's calibrated confidence per path to weight votes rather than treat all paths equally

Key Limitations



- Assumes the correct answer is more "reachable" via diverse reasoning than incorrect ones — this can fail on adversarial or genuinely ambiguous problems
- Not useful when there's no well-defined ground truth answer to vote on (e.g., creative writing, subjective judgments)
- Compute-hungry — a real constraint in production agentic systems with cost/latency budgets
- Doesn't fix systematic bias — if the model has a consistent blind spot or misconception, all N samples may reproduce the same wrong answer (voting won't help against correlated errors)

Self Consistency vs Reflection

Dimension	Self-Consistency	Reflection
Primary goal	Reduce variance / improve accuracy	Improve quality / fix errors
Mechanism	Sample N paths → vote	Generate → critique → revise
Number of passes	Multiple *parallel* samples	Multiple *sequential* iterations
Direction	Horizontal (breadth)	Vertical (depth)
Key operation	Aggregation (majority vote)	Critique + revision
Changes the answer?	Selects best from candidates	Transforms the answer
Needs a critic?	No	Yes (self or external)
Needs sampling?	Yes (temperature > 0)	Not necessarily
Compute cost	K× per query (parallel)	N× per query (sequential)
Latency	Parallelizable	Inherently sequential
Determinism	Reduces non-determinism	Can increase it (more rounds = more drift)
Best for	Discrete answers (math, logic, classification)	Open-ended outputs (code, writing, plans)
Weakness	Can't fix a systematically wrong model	Can loop, over-critique, or drift